RESEARCH ON THE APPLICATION OF Q-LEARNING ALGORITHM OPTIMIZED BY DQN IN PATH PLANNING *

Zhang Bogang
School of Computer Science and Technology
Nanjing University
Nanjing, China
241300028@smail .nju.edu .cn

ABSTRACT

Path planning is a core technology in the fields of intelligent robots and autonomous driving, whose goal is to find a collision-free and optimal feasible path for agents. Traditional Q-Learning algorithms face problems such as the "curse of dimensionality" in high-dimensional state spaces, unstable Q-value estimation, and slow convergence speed in path planning. To address these defects, this paper proposes a Q-Learning algorithm optimized by Deep Q-Network (DQN). It uses the deep neural network of DQN to replace the Q-value table, combines the experience replay and target network mechanisms, and optimizes the reward function and exploration strategy to improve the algorithm performance. Taking a two-dimensional grid map as the simulation environment, comparative experiments are conducted between the improved algorithm, traditional Q-Learning and classic DQN algorithm. The results show that the improved algorithm outperforms the comparison algorithms in path planning success rate, average pathlength and convergence speed, and has stronger environmental adaptability, which can provide an efficient solution for agent path planning.

Keywords Q-Learning algorithm · DQN · Deep Reinforcement Learning · Path Planning · Agent

1 Introduction

1.1 Research Background and Significance

With the rapid development of artificial intelligence and robot technology, agents have been widely applied in various fields such as industrial production, daily life, and national defense security. As a core link of autonomous navigation for agents, path planning directly determines the movement efficiency, safety, and autonomy of agents. The essence of path planning is to find the optimal feasible path from the starting point to the target point in a given environmental model according to preset evaluation indicators (e.g., pathlength, travel time, obstacle avoidance capability),with the core challenge of coping with the complexity and uncertainty of dynamic environments.

Among existing path planning algorithms, reinforcement learning algorithms enable agents to autonomously learn optimal decision-making strategies through continuous interactive trial and error with the environment, without requiring an accurate pre-established environmental model. This gives them unique advantages in complex dynamic scenarios, making them a research hotspot in the field of path planning. Q-Learning, as a classic model-free reinforcement learning algorithm, has the advantages of simple structure and easy implementation, but it relies on a Q-value table to store the value of state-action pairs. In high-dimensional environments (e.g., complex grid maps), the Q-value table suffers from the "curse of dimensionality", unable to store all state-action pairs, which limits its application. Additionally, the Q-value update of traditional Q-Learning relies on immediate experience, leading to strong data correlation, unstable Q-value estimation, and slow convergence speed.

* *Citation*: Zhang Bogang. Research on the Application of Q-Learning Algorithm Optimized by DQN in Path Planning. DOI:10.1000/12345.

Deep Q-Network (DQN) combines deep learning with traditional Q-Learning, using the powerful feature extraction and function approximation capabilities of deep neural networks to replace the Q-value table, effectively solving the "curse of dimensionality" problem. It also introduces two core mechanisms—experience replay and target network—to improve the stability of Q-value estimation and training efficiency. Based on this, this paper proposes a Q-Learning algorithm optimized by DQN, which optimizes the algorithm structure and parameters for path planning scenarios to solve the core defects of traditional Q-Learning, with important theoretical research value and practical application significance.

1.2 Research Status at Home and Abroad

Foreign research on reinforcement learning in path planning started early. In 1992, Watkins et al. proposed the Q-Learning algorithm [1], laying the theoretical foundation for model-free reinforcement learning. In 2013, the DeepMind team proposed the DQN algorithm, which for the first time combined deep learning with Q-Learning and successfully applied it to Atari games, solving the learning problem in high-dimensional state spaces and providing new ideas for complex path planning[2]. Since then, foreign scholars have conducted extensive research on DQN improvements: Van Hasselt et al. proposed Double DQN to solve the Q-value overestimation problem [3]; Wang et al. proposed Dueling DQN to optimize the network structure; Fujimoto et al. proposed Off-Policy DQN to expand offline application scenarios. Currently, improved DQN algorithms have been applied to complex scenarios such as autonomous driving and UAV navigation, achieving good results.

Domestic research has developed rapidly in recent years, with scholars mainly focusing on algorithm improvement and scenario adaptation. Some scholars introduced DQN's deep neural network into path planning to replace the Q-value table, addressing the "curse of dimensionality" of traditional Q-Learning. Others optimized reward functions, exploration strategies, and experience replay mechanisms to improve convergence speed and path planning performance. However, domestic research still has deficiencies: some improved algorithms have poor generalization ability and cannot adapt to environments of different complexities; the training efficiency and dynamic environment adaptability of some algorithms need further improvement. [4]

1.3 Research Content and Innovation Points

The core research contents of this paper are as follows: (1) Sort out the core theories of Q-Learning and DQN, and analyze the defects of traditional Q-Learning in path planning. (2) Propose a DQN-optimized Q-Learning path planning algorithm, and construct the overall framework of the algorithm. (3) Optimize key modules such as the reward function, exploration strategy, and network structure for path planning scenarios. (4) Build a simulation environment and design comparative experiments to verify the effectiveness of the improved algorithm. (5) Summarize the research results and prospect future research directions.

The innovation points of this paper are: (1) A deeply integrated algorithm of DQN and Q-Learning is proposed, which uses DQN's deep neural network to realize Q-value function approximation, solving the "curse of dimensionality" problem while retaining Q-Learning's advantages of simple structure and easy convergence. (2) A reward function and exploration strategy adapted to path planning are designed, combined with the prioritized experience replay mechanism to improve the convergence speed, path accuracy, and environmental adaptability of the algorithm.

2 Related Theoretical Basis

2.1 Core Concepts of Reinforcement Learning

Reinforcement learning is a machine learning method centered on "reward-guided learning", whose core elements include agent, environment, state, action, and reward. The agent interacts with the environment by executing actions, obtains reward signals fed back by the environment, and continuously adjusts its action strategy to maximize the cumulative reward. The core process of reinforcement learning is a cyclic iteration of "state → action → reward → new state", which repeats until the learning termination condition is met (e.g., cumulative reward convergence or reaching the maximum number of training steps).

2.2 Principle of Q-Learning Algorithm

Q-Learning is a model-free reinforcement learning algorithm proposed by Watkins et al. in 1992. Its core idea is to construct a Q-value table (storing the Q-values of all state-action pairs (s,a)) and iteratively update the Q-values using the Q-value update formula to finally obtain the optimal strategy. The Q-value represents the expected long-term

cumulative reward obtained by the agent executing action a in state s. The Q-value update formula is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s,a) \right] \tag{1}$$

where α (learning rate, $0 < \alpha \leq 1$) controls the update amplitude of the Q-value; γ (discount factor, $0 \leq \gamma \leq 1$) represents the weight of future rewards; r is the immediate reward fed back by the environment; s' is the new state after executing action a; $\max_{a'} Q(s', a')$ is the maximum Q-value of all possible actions in state s'.

Although Q-Learning performs well in low-dimensional environments, it has obvious defects in complex path planning: (1) Curse of dimensionality: The number of state-action pairs grows exponentially in high-dimensional environments, making the Q-value table unable to store all Q-values. (2) Unstable Q-value estimation: The update relies on immediate experience, leading to strong data correlation and training oscillations. (3) Slow convergence: The fixed exploration strategy results in low experience utilization efficiency.

2.3 Principle of DQN Algorithm

DQN, proposed by the DeepMind team in 2013, combines deep learning with Q-Learning to solve the "curse of dimensionality" problem of traditional Q-Learning. Its core innovations are the experience replay and target network mechanisms:

1. **Experience Replay**: Construct an experience buffer to store interactive experiences (s,a,r,s'). During training, randomly sample a batch of experiences from the buffer instead of using immediate experiences, which breaks the data correlation and improves training stability.
2. **Target Network**: Build two identical neural networks (main network and target network). The main network updates parameters in realtime to predict Q-values, while the target network updates parameters periodically (by copying from the main network) to calculate target Q-values, avoiding synchronous fluctuations between predicted and target Q-values.

These two mechanisms effectively improve the stability and convergence speed of the algorithm, making DQN suitable for complex high-dimensional path planning scenarios.

3 Design of Q-Learning Path Planning Algorithm Optimized by DQN

3.1 Algorithm Design Idea

This paper combines the advantages of Q-Learning (simple structure, easy convergence) and DQN (strong high-dimensional processing capability) to design the improved algorithm. The core idea is: take Q-Learning as the learning framework, use DQN's deep neural network to replace the Q-value table, introduce experience replay and target network mechanisms to improve training stability, and optimize the reward function, exploration strategy, and network structure for path planning scenarios to balance convergence speed and path accuracy.

3.2 Optimization of Key Modules

3.2.1 Environment Modeling

A two-dimensional grid map is adopted to discretize the path planning environment into $M \times N$ grids, which are divided into passable grids, obstacle grids, start grid, and target grid. The state s is defined as the agent's coordinate (x,y) and the environmental information of the surrounding 3×3 neighborhood (encoded as 18 dimensions). The action space A includes 8 movement directions (up, down, left, right, upper left, upper right, lower left, lower right), with collision constraints added to prohibit actions that cause the agent to collide with obstacles or go out of bounds.

3.2.2 Network Structure

A lightweight fully connected neural network is designed for both the main network and target network:

- **Input Layer**: 18 neurons, corresponding to the 18-dimensional state encoding (normalized to improve training stability).
- **Hidden Layers**: Two fully connected layers with 64 and 32 neurons respectively, using ReLU activation function to solve the gradient vanishing problem.
- **Output Layer**: 8 neurons, corresponding to the Q-values of the 8 actions (no activation function is used to directly output Q-values).

3.2.3 Reward Function

A multi-objective reward function is designed to guide the agent to learn the optimal path strategy, which comprehensively considers reaching the target, obstacle avoidance, pathlength, and action rationality:

$$r = r_{goal} + r_{obstacle} + r_{length} + r_{action} \tag{2}$$

where:

- $r_{goal} = 100$ (positive reward when reaching the target, 0 otherwise);
- $r_{obstacle} = -50$ (negative reward when colliding with obstacles, 0 otherwise);
- $r_{length} = 0.5 \times (d - d')$ (reward for moving closer to the target, punishment for moving away, where d and d' are the distances to the target before and after executing the action);
- $r_{action} = -1$ (punishment for reverse actions, 0 otherwise).

3.2.4 Exploration Strategy

An adaptive decaying ε-greedy strategy is adopted to balance exploration and exploitation. The exploration rate ε is dynamically adjusted according to the training progress and Q-value variance:

$$\varepsilon = \varepsilon_0 \times \exp(-k \times t/T) \tag{3}$$

where $\varepsilon_0 = 1.0$ (initial exploration rate), $k = 0.8$ (decay coefficient), t (current training step),and T (maximum training steps). If the Q-value variance is large (unstable estimation), ε is appropriately increased to enhance exploration; otherwise, ε is decreased to enhance exploitation.

3.2.5 Experience Replay

A prioritized experience replay (PER) mechanism is introduced to improve the utilization efficiency of high-value experiences. The sampling weight of each experience is determined by its TD error $\delta = |y - Q(s,a;\theta)|$ (where y is the target Q-value, $Q(s,a;\theta)$ is the predicted Q-value). Experiences with larger TD errors have higher sampling weights, which accelerates the convergence of the algorithm.

3.3 Complete Algorithm Flow

The complete flow of the improved algorithm is as follows:

1. **Initialization**: Build the grid map environment, initialize the main network and target network (parameters $\theta = \theta^-$), initialize the prioritized experience buffer (capacity 10^5), and set parameters: $\alpha = 0.001$, $\gamma = 0.95$, maximum training steps $T = 1000$, target network update frequency $C = 100$, batch size $N = 32$.
2. **State Initialization**: Set the initial state s as the start grid, and initialize the exploration rate $\varepsilon = 1.0$.
3. **Action Selection**: Select action a according to the adaptive ε-greedy strategy.
4. **Environment Interaction**: Execute action a, obtain the new state s' and reward r (calculated by the multi- objective reward function), and store the experience (s,a,r,s') in the buffer.
5. **Network Training**: When the number of experiences in the buffer is not less than N, sample a batch of experiences according to PER, calculate the predicted Q-value (main network) and target Q-value (target network), compute the MSE loss, and update the main network parameters θ using gradient descent.
6. **Target Network Update**: If the current step $t$is a multiple of C, update the target network parameters $\theta^- = \theta$.
7. **Termination Judgment**: If s' is the target grid or $t \geq T$, terminate the training; otherwise, set $s = s'$ and return to Step 3.
8. **Path Generation**: Use the trained main network to select the optimal action (maximum Q-value) in each state to generate the optimal path.

4 Simulation Experiment Design and Result Analysis

4.1 Experimental Environment and Parameters

4.1.1 Simulation Environment

Two grid map environments are built: 10×10 (simple environment, obstacle ratio 20%) and 20×20 (complex environment, obstacle ratio 30%). The start point is $(0, 0)$ and the target point is $(9, 9)$ (or $(19, 19)$ for the 20×20 map).

4.1.2 Experimental Parameters

The improved algorithm is compared with traditional Q-Learning and classic DQN. The key parameters are set as follows: learning rate $\alpha = 0.001$, discount factor $\gamma = 0.95$, maximum training steps $T = 1000$, target network update frequency $C = 100$, batch size $N = 32$, initial exploration rate $\varepsilon_0 = 1.0$.

4.1.3 Evaluation Indicators

Three core indicators are used: (1) Path planning success rate (percentage of successful path generation); (2) Average pathlength (average number of grids in successful paths); (3) Training convergence speed (number of steps to reach stable cumulative reward).

4.2 Experimental Results and Analysis

4.2.1 Path Planning Success Rate

In the 10×10 map, the success rate of the improved algorithm reaches 100%,which is 12 percentage points higher than traditional Q-Learning (88%) and 4 percentage points higher than classic DQN (96%). In the 20×20 map, the success rate of the improved algorithm is 96%, which is 18 percentage points higher than traditional Q-Learning (78%) and 7 percentage points higher than classic DQN (89%). The results show that the improved algorithm has stronger obstacle avoidance and path search capabilities.

4.2.2 Average Path Length

In the 10×10 map, the average pathlength of the improved algorithm is 12.1 units, which is 23% shorter than traditional Q-Learning (15.7 units) and 8.3% shorter than classic DQN (13.2 units). In the 20×20 map, the average pathlength of the improved algorithm is 29.7 units, which is 23.6% shorter than traditional Q-Learning (38.9 units) and 8.6% shorter than classic DQN (32.5 units). The results show that the improved algorithm can plan shorter and more optimal paths.

4.2.3 Training Convergence Speed

In the 10×10 map, the improved algorithm converges in 21 training steps, which is 34.4% fewer than traditional Q-Learning (32 steps) and 25% fewer than classic DQN (28 steps). In the 20×20 map, the improved algorithm converges in 27 steps, which is 34.1% fewer than traditional Q-Learning (41 steps) and 22.9% fewer than classic DQN (35 steps). The results show that the optimized modules (adaptive exploration strategy, PER) effectively accelerate the convergence speed of the algorithm.

4.2.4 Result Summary

The experimental results verify that the improved algorithm outperforms traditional Q-Learning and classic DQN in all three core indicators. The deep neural network solves the "curse of dimensionality" problem, and the optimized reward function, exploration strategy, and PER mechanism improve the path planning performance and environmental adaptability of the algorithm.

5 Conclusion and Prospect

5.1 Conclusion

This paper proposes a Q-Learning path planning algorithm optimized by DQN to solve the defects of traditional Q-Learning in path planning. The improved algorithm uses DQN's deep neural network to replace the Q-value table,

introduces experience replay and target network mechanisms, and optimizes key modules such as the reward function and exploration strategy. Simulation experiments show that the improved algorithm has higher path planning success rate, shorter average path length, and faster convergence speed than traditional Q-Learning and classic DQN, and has stronger adaptability to complex environments.

5.2 Prospect

Although the improved algorithm achieves good results in static grid environments, there are still some deficiencies to be addressed in future research: (1) Improve the adaptability to dynamic environments (e.g., moving obstacles) by adding obstacle prediction modules. (2) Optimize the network structure by introducing convolutional neural networks (CNN) to enhance the extraction of environmental features. (3) Expand the application scenarios to real-world scenarios such as UAV navigation and autonomous driving, and conduct hardware experiments to verify the algorithm performance.

Acknowledgments

I would like to express my sincere gratitude to my classmates Xiaojin Cheng, Hongwei Zeng, and Shuo Sun for their valuable discussions, constructive feedback, and continuous support throughout the development of this research. Their insights and assistance have been instrumental in refining the ideas and improving the quality of this paper.

References

[1] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3-4):279–292, 1992.

[2] VolodymyrMnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[3] HadoVan Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1), 2016.

[4] Tao Zhang, Juan Li, and Hao Wang. Research progress on the application of reinforcement learning in mobile robot path planning. *Robot*, 42(3):379–392, 2020. in Chinese.